

Algoritmi-Modulo algoritmi per bioinformatica

Divide et impera-Capitolo 4 [CLRS]

Roberto Posenato

versione 2026-2, 23/03/2026

- 1 Divide et impera
 - Idea generale
- 2 Dettagli
 - Pseudocodifica
 - Considerazioni
 - Costo computazionale
- 3 Esempi di applicazione
 - Prodotto
 - MergeSort
 - Massimo SottoArray
- 4 Equazioni di ricorrenza
- 5 Altri problemi risolvibili

*“Tell me and I forget, teach me and I remember,
involve me and I learn.”*

Benjamin Franklin

*“A student who achieves knowledge through free investigation
and spontaneous effort will be able to retain that knowledge
and will have acquired a methodology that can serve for a li-
fetime.”*

Jean Piaget

Scopo della lezione

- Introdurre la tecnica di progettazione *divide et impera* mediante l'analisi di tre algoritmi.
- Ripassare delle *equazioni di ricorrenza algoritmiche* per la rappresentazione e la determinazione della complessità degli algoritmi *divide et impera*.
- Ripassare le tre principali tecniche per risolvere le equazioni di ricorrenza.

Nota!

- Nel capitolo 4 [CLRS] vengono presentati anche degli algoritmi *divide et impera* per la moltiplicazione di matrici (non in programma).
- In queste lezioni considereremo algoritmi più semplici.

Idea generale

Il paradigma *divide et impera* prevede di risolvere un problema eseguendo i seguenti passi:

- 1 **Suddividere** il problema in sottoproblemi di dimensioni inferiori;
- 2 **Risolvere ricorsivamente** i sottoproblemi;
- 3 **Combinare** i risultati ottenuti in modo appropriato per determinare la soluzione;

Aspetti cruciali per una buona applicazione:

- **Suddivisione:** deve determinare sottoproblemi il più possibile disgiunti;
- **Ricorsione:** deve terminare quando la dimensione permette una risoluzione diretta;
- **Combinazione:** non deve richiedere un tempo maggiore di quello del calcolo delle soluzioni dei sottoproblemi.

Il paradigma *divide et impera* può essere così schematizzato:

Commento:

Algoritmo: DIVIDEETIMPERA(X)

Input: X istanza di dimensione $|X| = n$

Output: Y , soluzione

1: **if** ($|X| \leq n_0$) **then** // n_0 = costante dell'alg.

2: **return** ADHOC(X)

3: **dividi** X in sottoistanze “più piccole”

$X_1, X_2, \dots, X_a$

4: **for** $i = 1$ **to** a **do**

5: $Y_i = \text{DIVIDEETIMPERA}(X_i)$

6: **combina** gli Y_i in una soluzione Y

7: **return** Y

- La funzione **ADHOC**(X):
 - calcola la soluzione in modo diretto;
 - complessità $\approx$ ordine lineare o costante.
- **dividi** *dovrebbe*:
 - suddividere l'istanza in sottoistanze “disgiunte” di dimensione n/b ciascuna, con $b > 1$;
 - avere complessità $\approx$ lineare
- Il numero di sottoistanze a :
 - solitamente piccolo: $[2, \dots, 4]$ e diverso da b .
 - $a = 1 \Rightarrow$ si chiama “semplificazione”.
- **combina** ha solitamente complessità $O(n^d)$ per un d costante.

Il costo algoritmo *divide et impera* = soluzione di un'equazione di ricorrenza:

- Sia $\text{tempo}(\text{ADHOC}(X)) = g(|X|)$;
- Sia $\text{tempo}(\text{dividi}(X)) + \text{tempo}(\text{combina}(Y_1, \dots, Y_a)) = f(|X|)$;
- $T(n) = \text{tempo}(\text{DIVIDEETIMPERA}(X))$. Posto $n = |X|$,

$$T(n) = \begin{cases} g(n) & n \leq n_0, \\ aT(n/b) + f(n) & \text{altrimenti.} \end{cases}$$

- Per la risoluzione di questa equazione si ricorre ai metodi: *di sostituzione, dell'albero di ricorsione o dell'esperto*.

Dettagli

Ricorsione o iterazione?

Ricorsione o iterazione?

È possibile sostituire la ricorsione con un ciclo iterativo.

Pro e contro

- Vantaggi:**
- si elimina l'overhead delle chiamate ricorsive;
 - si può risparmiare fino a $\Theta(h)$ spazio ausiliario, dove h è la profondità della ricorsione.

Svantaggi: l'algoritmo diviene un po' più complicato da scrivere.

Esempi di applicazione

Prodotto

PROBLEMA PRODOTTO NUMERI

INPUT: Due interi x, y di n bit ciascuno.

OUTPUT: Determinare il loro prodotto.

- Il costo del prodotto tradizionale = $O(n^2)$;

Nota!

Il costo si riferisce al caso in cui si debbano moltiplicare due numeri con $n > 64$ bit usando il metodo tradizionale.

Se $n \leq 64$, i moderni calcolatori permettono il calcolo diretto!

Esempi di applicazione

Prodotto

Si può fare meglio?

- Assumiamo, per ora, che $n = 2^k$.
- Si suddivide $x = \boxed{x_L} \boxed{x_R} = 2^{\frac{n}{2}} x_L + x_R$.
- Lo stesso per $y = 2^{\frac{n}{2}} y_L + y_R$;
- Quindi il prodotto può essere espresso in termini più piccoli come:

$$\begin{aligned} xy &= (2^{\frac{n}{2}} x_L + x_R) \times (2^{\frac{n}{2}} y_L + y_R) \\ &= 2^n (x_L \times y_L) + 2^{\frac{n}{2}} (x_L \times y_R + x_R \times y_L) + (x_R \times y_R) \end{aligned}$$

Esempio 1

$x = 0000\ 0110$, $y = 0001\ 0101$, $xy = 0111\ 1110$

$x_L = 0000$, $x_R = 0110$, $y_L = 0001$, $y_R = 0101$

$$\begin{aligned} xy &= 2^n (x_L \times y_L) + 2^{\frac{n}{2}} (x_L \times y_R + x_R \times y_L) + x_R \times y_R = \\ &= (0000 \times 0001) \ll 8 + (0000 \times 0101 + 0110 \times 0001) \ll 4 + 0110 \times 0101 = \\ &= 0000 + (0000 + 0110) \ll 4 + 0001\ 1110 = 0110\ 0000 + 0001\ 1110 = 0111\ 1110 \end{aligned}$$

Esempi di applicazione

Prodotto

Si può fare meglio?

- Il prodotto

$$xy = 2^n(x_L \times y_L) + 2^{\frac{n}{2}}(x_L \times y_R + x_R \times y_L) + (x_R \times y_R)$$

richiede **4 moltiplicazioni** (su fattori di dimensione $\frac{n}{2}$) + **3 addizioni** (su addendi di dimensione lineare) + **2 shift** (su termini di dimensione lineare).

- Se si risolve ricorsivamente ciascuna moltiplicazione fino a quando la dimensione n diventa 1, si ha uno schema *divide et impera*
- Complessità in tempo $T(n) = 4T(\frac{n}{2}) + O(n) = O(n^2) \Leftarrow$ **Tutto inutile?**

Esempi di applicazione

Prodotto

Gauss (1777–1855) ha osservato che il prodotto di 2 numeri complessi

$$(a + bi) \times (c + di) = a \times c - b \times d + (b \times c + a \times d)i$$

può essere eseguito con 3 moltiplicazioni anziché 4:

$$a \times c, b \times d, (a + b) \times (c + d)$$

dato che $b \times c + a \times d = (a + b) \times (c + d) - a \times c - b \times d$.

Esempi di applicazione

Prodotto

- Applicando la stessa tecnica al prodotto di xy , si ottiene che è sufficiente calcolare

$$x_L \times y_L, \quad x_R \times y_R, \quad (x_L + x_R) \times (y_L + y_R)$$

- Infatti

$$\begin{aligned} xy &= 2^n(x_L \times y_L) + 2^{\frac{n}{2}}(x_L \times y_R + x_R \times y_L) + (x_R \times y_R) \\ &= 2^n(x_L \times y_L) + 2^{\frac{n}{2}}((x_R + x_L) \times (y_R + y_L) - (x_L \times y_L) - (x_R \times y_R)) + (x_R \times y_R) \end{aligned}$$

Esempio 2

$$x = 0000\ 0110, y = 0001\ 0101, xy = 0111\ 1110$$

$$x_L = 0000, x_R = 0110, y_L = 0001, y_R = 0101$$

$$\begin{aligned} xy &= 2^n(x_L \times y_L) + 2^{\frac{n}{2}}((x_R + x_L) \times (y_R + y_L) - (x_L \times y_L) - (x_R \times y_R)) + (x_R \times y_R) \\ &= 0000 + (0110 \times 0110 - 0000 - 0001\ 1110) \ll 4 + 0001\ 1110 = \\ &= 0000 + (0010\ 0100 - 0001\ 1110) \ll 4 + 0001\ 1110 = \\ &0110 \ll 4 + 0001\ 1110 = 0111\ 1110 \end{aligned}$$

Esempi di applicazione

Algoritmo per il prodotto

Algoritmo: PRODOTTO(x, y)

Input: x, y due interi non negativi di dimensione $n = 2^l$

Output: prodotto xy

- 1: **if** $n \leq 64$ **then return** xy // caso base. $\Theta(1)$
 - 2: $x_L = \frac{n}{2}$ bit più a sx, $x_R = \frac{n}{2}$ bit più a dx di x // Divide. $\Theta(n)$
 - 3: $y_L = \frac{n}{2}$ bit più a sx, $y_R = \frac{n}{2}$ bit più a dx di y
 - 4: $P_1 = \text{PRODOTTO}(x_L, y_L)$ // Impera. $3T(\frac{n}{2}) + \Theta(n)$
 - 5: $P_2 = \text{PRODOTTO}(x_R, y_R)$
 - 6: $P_3 = \text{PRODOTTO}(x_L + x_R, y_L + y_R)$
 - 7: $prod = (P_1 \ll n) + ((P_3 - P_1 - P_2) \ll \frac{n}{2}) + P_2$ // Combina. $\Theta(n)$
 - 8: **return** $prod$
-

Complessità

$$T(n) = 3T\left(\frac{n}{2}\right) + \Theta(n) = \Theta(n^{\lg 3}) = \Theta(n^{1.59})$$

Nota: la soluzione si determina facilmente con il metodo dell'esperto.

Esempi di applicazione

MERGESORT

MERGESORT è un algoritmo di ordinamento già introdotto in una precedente lezione.

Lo schema *divide et impera* è immediato:

- **Divide:** l'array di n elementi viene visto come 2 sottoarray di $n/2$ elementi.
- **Impera:** ordina i due sottoarray in modo ricorsivo (quando l'array ha dimensione 1, la ricorsione finisce).
- **Combina:** **fonde** i due sottoarray ordinati.

Operazione chiave è quella di **fondere** i due sottoarray ordinati.

Esempi di applicazione

MERGESORT: analisi di complessità

- Inizialmente si assume $n = 2^l$
- Si considerano $a = 2$ sottoistanze di dimensione $\frac{n}{2}$ ciascuna;
- Per dividere logicamente l'istanza in 2 serve tempo $D(n) = \Theta(n)$ (l'algoritmo deve creare i due sottoarray);
- Per fondere le soluzioni serve tempo $C(n) = \Theta(n)$;
- Quindi, $\Theta(n) + \Theta(n) = \Theta(n)$,

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 2T(\frac{n}{2}) + \Theta(n) & n > 1 \end{cases}$$

- $T(n) = \Theta(n \lg n)$.

Esempi di applicazione

Massimo SottoArray

Consideriamo il problema del massimo sotto array*.

PROBLEMA MASSIMO SOTTOARRAY

INPUT: Un array di interi $A[1 : n]$

OUTPUT: Determinare un sotto array non vuoto i cui valori hanno somma massima:

$$\max_{1 \leq i \leq j \leq n} A[i] + A[i + 1] + \dots + A[j].$$

*Questo problema è presente **solo** nella III edizione [CLRS]

Esempi di applicazione

Massimo SottoArray

Esempio 3

Profitti di una società negli anni 2001–2012:

Anno	2001	2002	2003	2004	2005	2006	2007	2008	2009	2010	2011	2012
Profitto M€	8	10	15	-4	-7	1	-25	32	-5	10	-10	4

Anno	2001	2002	2003	2004	2005	2006	2007	2008	2009	2010	2011	2012
Profitto M€	8	10	15	-4	-7	1	-25	32	-5	10	-10	4

Anno	2001	2002	2003	2004	2005	2006	2007	2008	2009	2010	2011	2012
Profitto M€	8	10	15	-4	-7	1	-25	32	-5	10	-10	4

Anno	2001	2002	2003	2004	2005	2006	2007	2008	2009	2010	2011	2012
Profitto M€	8	10	15	-4	-7	1	-25	32	-5	10	-10	4

- Dal 2001 al 2003 il profitto è stato di $3 + 10 + 15 = 33$.
- Dal 2006 al 2010 il profitto è stato di $1 - 25 + 32 - 5 + 10 = 13$.
- Dal 2008 al 2010 il profitto è stato di $32 - 5 + 10 = 37$

Esempi di applicazione

Massimo SottoArray

Soluzione *forza bruta*

- $somma = -\infty$;
- Per tutte le coppie possibili (inizio, fine):
 - se la somma elementi di $A[inizio: fine] > somma$, aggiorna somma e ricorda (inizio, fine)
- Stampa somma e coppia (inizio, fine) salvata.
- Complessità:
 - Se $|A| = n$, ci sono $\binom{n+1}{2} = \frac{n(n+1)}{2} = \Theta(n^2)$ coppie (*inizio*, *fine*) con $inizio \leq fine$.
 - Per ciascuna coppia, il calcolo della somma richiede $O(n)$ passi.
 - Quindi la complessità in tempo è $\Theta(n^3)$ nel caso peggiore.
 - Si può anche dire che complessità in tempo $\Omega(n^2)$.

Esempi di applicazione

Massimo SottoArray

Soluzione mediante *divide et impera*

- Il metodo suggerisce di dividere l'istanza $A[low : high]$ in due parti possibilmente uguali.
- Quindi, si pone $mid = (low + high)/2$ e si considero le due sottoistanze $A[low : mid]$ e $A[mid + 1 : high]$
- Una soluzione (sol) del problema deve essere quindi presente esattamente in una delle seguenti:
 - 1 Interamente in $A[low : mid]$
 - 2 Interamente in $A[mid + 1 : high]$
 - 3 A cavallo di mid , quindi in $A[i : j]$ con $low \leq i \leq mid < j \leq high$.
- $sol: \max\{sol(A[low : mid]), sol(A[i : j]), sol(A[mid + 1 : high])\}$

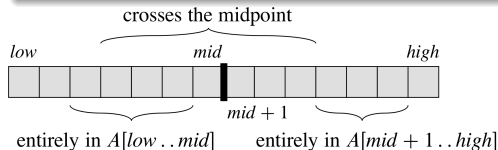
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
8	10	15	-4	-7	1	-25	32	-5	10	-30	4	-3	20	-1

Esempi di applicazione

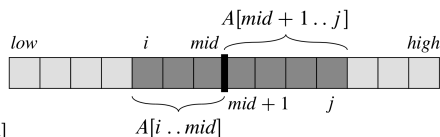
Massimo SottoArray

Soluzione mediante *divide et impera*

- Si risolve ricorsivamente il problema considerando $A[low : mid]$ e $A[mid + 1 : high]$ (sono istanze più piccole).
Ci si ferma quando l'istanza ha dimensione 1 (la soluzione è immediata)!
- Invece, per trovare la somma massima per il sotto array $A[low : high]$ che contiene il punto mid , si può:
 - 1 Calcolare la somma massima partendo da mid e scendendo fino a low
Si supponga che il massimo è dato da $\sum A[i : mid]$ dove $i \geq low$.
 - 2 Calcolare la somma massima partendo da $mid + 1$ e salendo fino a $high$
Si supponga che il massimo è dato da $\sum A[mid + 1 : j]$ dove $j \leq high$.
 - 3 Restituire $\sum A[i : mid] + \sum A[mid + 1 : j]$



(a)



(b)

Esempi di applicazione

Massimo SottoArray

Questa è la procedura che determina il sotto array con somma massima che passa per il punto di mezzo.

FIND-MAX-CROSSING-SUBARRAY ($A, low, mid, high$)

// Find a maximum subarray of the form $A[i \dots mid]$.

$left\text{-}sum = -\infty$

$sum = 0$

for $i = mid$ **downto** low

$sum = sum + A[i]$

if $sum > left\text{-}sum$

$left\text{-}sum = sum$

$max\text{-}left = i$

// Find a maximum subarray of the form $A[mid + 1 \dots j]$.

$right\text{-}sum = -\infty$

$sum = 0$

for $j = mid + 1$ **to** $high$

$sum = sum + A[j]$

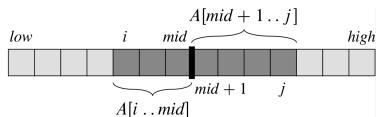
if $sum > right\text{-}sum$

$right\text{-}sum = sum$

$max\text{-}right = j$

// Return the indices and the sum of the two subarrays.

return ($max\text{-}left, max\text{-}right, left\text{-}sum + right\text{-}sum$)



Esempi di applicazione

Massimo SottoArray

Complessità FIND-MAX-CROSSING-SUBARRAY

- Se il sottoarray $A[low : high]$ contiene $m = high - low + 1$ elementi, il numero di cicli complessivo dei due **for** è m .
- Ogni ciclo esegue un numero costante di passi.
- Complessità della procedura = $\Theta(m)$.

Esempi di applicazione

Massimo SottoArray

Questa è la procedura principale *divide et impera* che risolve il problema: è sufficiente invocare **FIND-MAXIMUM-SUBARRAY($A, 1, A.length$)**.

FIND-MAXIMUM-SUBARRAY($A, low, high$)

if $high == low$

return ($low, high, A[low]$)

else $mid = \lfloor (low + high) / 2 \rfloor$

 ($left-low, left-high, left-sum$) =

FIND-MAXIMUM-SUBARRAY(A, low, mid)

 ($right-low, right-high, right-sum$) =

FIND-MAXIMUM-SUBARRAY($A, mid + 1, high$)

 ($cross-low, cross-high, cross-sum$) =

FIND-MAX-CROSSING-SUBARRAY($A, low, mid, high$)

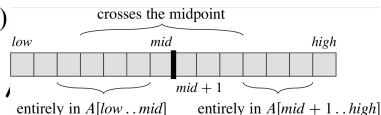
if $left-sum \geq right-sum$ and $left-sum \geq cross-sum$

return ($left-low, left-high, left-sum$)

elseif $right-sum \geq left-sum$ and $right-sum \geq cross-sum$

return ($right-low, right-high, right-sum$)

else return ($cross-low, cross-high, cross-sum$)



Esempi di applicazione

Massimo SottoArray

Complessità FIND-MAXIMUM-SUBARRAY

- Si assuma, per ora, che l'input sia $A[1 : n]$ con $n = 2^k$.
- Sia $T(n)$ la complessità di FIND-MAXIMUM-SUBARRAY.
- Caso base ($n = 1$) richiede tempo costante: $T(n) = \Theta(1)$
- Caso generale ($n > 1$), la complessità è data da
 - la complessità di suddividere l'istanza in due sottoistanze: $\Theta(1)$;
 - più la complessità dei sottoproblemi: $2T(n/2)$;
 - più la complessità di FIND-MAX-CROSSING-SUBARRAY su $A[low : high]$: $\Theta(n)$.
- Complessità è quindi rappresentabile come equazione di ricorrenza:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 2T(n/2) + \Theta(1) + \Theta(n) & \text{se } n > 1 \end{cases}$$

- Applicando il metodo dell'esperto, $T(n) = \Theta(n \lg n)$.

Equazioni di ricorrenza

Ripasso

Formato

Un'equazione di ricorrenza **algoritmica** $T(n)$ è definita come

$$T(n) = \begin{cases} K & \text{se } n \leq k \\ aT(n/b) + C(n) & \text{altrimenti} \end{cases}$$

dove n è la variabile indipendente, K, k, a, b sono costanti positive e $C(n)$ è una funzione di n .

- Risolvere un'equazione di ricorrenza significa determinare una formula chiusa per $T(n)$ (Esempio: $T(n) = n^2$).
- Esistono 3 metodi principali per risolvere un'equazione di ricorrenza:
 - 1 Metodo di sostituzione.
 - 2 Metodo dell'albero di ricorsione.
 - 3 Metodo dell'esperto.

Equazioni di ricorrenza

Metodo di sostituzione

Prevede:

- Ipotizzare una soluzione.
- Usare l'induzione matematica per trovare le costanti e dimostrare che la soluzione è corretta.

Nota!

- Si applica quando si può facilmente intuire una possibile soluzione.
- Si deve prestare attenzione che la dimostrazione per induzione determini esattamente la stessa soluzione ipotizzata.

Equazioni di ricorrenza

Metodo di sostituzione: esempio corretto

Si deve risolvere:

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 2T(\lfloor \frac{n}{2} \rfloor) + n & \text{altrimenti} \end{cases}$$

- Ipotesizzare una soluzione: $T(n) = O(n \lg n)$
- Si deve dimostrare quindi che, per una qualche coppia di costanti $c > 0, n_0 > 1$, $T(n) \leq cn \lg n$ per tutti gli $n \geq n_0$.
- Passo induttivo: si assume vero per $2n_0 \leq n' < n$. Per esempio, si assume vero per $n' = \lfloor \frac{n}{2} \rfloor$: $T(\lfloor \frac{n}{2} \rfloor) \leq c \lfloor \frac{n}{2} \rfloor \lg \lfloor \frac{n}{2} \rfloor$.

Nota!

In [CLRS] si analizza il caso con $2T(\lfloor \frac{n}{2} \rfloor) + \Theta(n)$.

Equazioni di ricorrenza

Metodo di sostituzione: esempio corretto

- Vale allora

$$\begin{aligned}T(n) &\leq 2(c \lfloor \frac{n}{2} \rfloor \lg \lfloor \frac{n}{2} \rfloor) + n \\&\leq cn \lg(\frac{n}{2}) + n \\&= cn \lg n - cn \lg 2 + n \\&\leq cn \lg n\end{aligned}$$

ultimo passo vale solo quando $c \geq 1$.

- Per completare la dimostrazione, si deve dimostrare che la soluzione vale anche per il caso base, $n_0 \leq n' < 2n_0$.
- Per $n_0 = 2$, con $c \geq 2$ si ha che vale $T(2) = 4 \leq c2 \lg 2$.
- Quindi la soluzione è valida a partire da $n_0 = 2$.

Equazioni di ricorrenza

Metodo di sostituzione: esempio con errore

- Quando si ipotizzano soluzioni con O o Θ , è facile sbagliare.
- Per esempio, per la precedente equazione, si potrebbe ipotizzare $T(n) = O(n)$.
- Quindi si deve dimostrare per induzione che $T(n) \leq cn$:

$$\begin{aligned}T(n) &\leq 2(c\lfloor \frac{n}{2} \rfloor) + n \\&\leq cn + n \\&= O(n) \Leftarrow \text{Errato!}\end{aligned}$$

- È errato perché si è dimostrato $T(n) \leq cn + n = (c + 1)n$ anziché $T(n) \leq cn$.

Equazioni di ricorrenza

Metodo dell'albero di ricorsione

Prevede:

- Disegnare l'albero delle chiamate ricorsive mettendo in ciascun nodo il costo della sottoistanza associata.
- Si sommano quindi i costi per ciascun livello dell'albero.
- Si sommano infine i costi di tutti i livelli.

Nota!

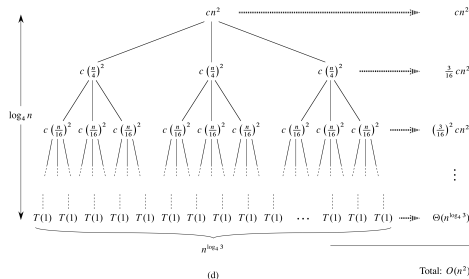
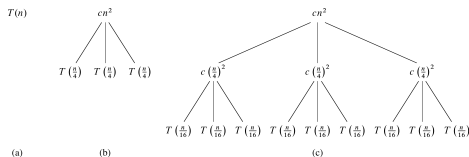
- Solitamente si applica con una certa approssimazione (sui costi) per ottenere un'ipotesi di soluzione da verificare con il metodo di sostituzione.
- Se, invece, si riesce a fare un'analisi precisa dei costi delle singole sottoistanze, la soluzione è dimostrata essere corretta già dall'albero.

Equazioni di ricorrenza

Metodo dell'albero di ricorsione: esempio

Si vuole trovare la soluzione all'equazione di ricorrenza

$T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$. L'albero della ricorsione è il seguente:



- Dimensione istanza al livello i : $\frac{n}{4^i}$
- Livello più basso: $\log_4 n$.
- Livelli: $(0, \dots, \log_4 n)$
- Ogni livello i ha 3^i nodi.
- Ogni nodo al livello i ha costo $c(\frac{n}{4^i})^2$
- Costo del livello $i < \log_4 n$:
 $3^i c(\frac{n}{4^i})^2 = (\frac{3}{16})^i cn^2$
- Ultimo livello ha $3^{\log_4 n} = n^{\log_4 3}$ nodi di costo costante.
- Sommando i costi di tutti i livelli (si approssima la somma con la serie geometrica): $< \frac{16}{13} cn^2 + \Theta(n^{\log_4 3})$
- Costo totale: $O(n^2)$

Equazioni di ricorrenza

Metodo dell'esperto

Teorema 1 (Teorema dell'esperto)

*Date le costanti $a > 0$, $b > 1$ e la funzione $f(n)$, detta **forzante** (asintoticamente positiva), la ricorrenza*

$$T(n) = aT(n/b) + f(n)$$

può essere limitata asintoticamente come:

- 1 Se $\exists \varepsilon > 0$ tale che $f(n) = O(n^{(\log_b a) - \varepsilon})$, $T(n) = \Theta(n^{\log_b a})$
- 2 Se $\exists k \geq 0$ t.c. $f(n) = \Theta(n^{\log_b a} \lg^k n)$, $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$
- 3 Se $\exists \varepsilon > 0$ t.c. $f(n) = \Omega(n^{(\log_b a) + \varepsilon})$ e $af(n/b) \leq cf(n)$ per almeno un $c < 1$ e $\forall n$ sufficientemente grande, $T(n) = \Theta(f(n))$

Nota!

$n^{\log_b a} = a^{\log_b n}$ è il numero di foglie dell'albero di ricorsione!

Il teorema assume che il costo di calcolo di un'istanza-foglia = $O(1)$.

Equazioni di ricorrenza

Metodo dell'esperto: esempi

- Caso semplice: $T(n) = 9T(n/3) + n$

Si ha $a = 9$, $b = 3$, $f(n) = n$. Quindi $n^{\log_b a} = n^{\log_3 9} = n^2$. Ora, visto che $f(n) = n = O(n^{2-\epsilon})$, si applica caso 1:

$$T(n) = \Theta(n^2).$$

- Caso 2: $T(n) = 2T(n/2) + n \lg n$

Si ha $a = 2$, $b = 2$, $f(n) = n \lg n$. Quindi $n^{\log_b a} = n^{\log_2 2} = n$.

Si può pensare di applicare il caso 2 con $k = 1$:

$$f(n) = \Theta(n^{\log_2 2} \lg^1 n) = \Theta(n \lg n) \text{ e quindi } T(n) = \Theta(n \lg^2 n).$$

- Si lascia per esercizio la dimostrazione di tutte le ricorrenze degli algoritmi visti fino a ora.

Altri problemi risolvibili con *divide et impera*

Elenco parziale di problemi che ammettono un buon algoritmo di risoluzione del tipo *divide et impera*.

Problema	Costo naïve	Costo <i>divide et impera</i>
Ricerca in un array ordinato	$O(n)$	ricerca binaria: $O(\log n)$
Ordinamento per confronto	$O(n^2)$	MERGESORT: $\Theta(n \log n)$ QUICKSORT: $\Theta(n^2)$ caso peggiore, $O(n \log n)$ caso medio.
Ricerca mediana	$\Theta(n \log n)$	$\Theta(n)$
Ricerca rank in un piano	$O(n^2)$	$O(n \log^2 n)$
Ricerca coppia più vicina in un piano	$O(n^2)$	$O(n \log n)$
Inviluppo convesso	$O(n^2)$	$O(n \log n)$

In questa lezione si è visto:

- Come è strutturato il metodo *divide et impera*.
- Applicazione del metodo ai problemi Massimo SottoArray, Prodotto di interi e MERGESORT.
- Cosa sono le equazioni di ricorrenza e quali sono i metodi usati per risolverle.

- Studiare il capitolo 4 [CLRS] escludendo i sottocapitoli 4.1, 4.2, 4.6 e 4.7.
- Scrivere lo pseudocodice dell'algoritmo *forza bruta* per il problema Massimo SottoArray.
- Risolvere esercizi 4.3–1a, 4.3–1b, 4.3–1c, 4.4–3, 4.5–4 [CLRS].

Esercizi

Equazioni di ricorrenza

Risolvere le seguenti equazioni di ricorrenza:

$$T(n) = 2T(n/3) + 1 \quad (1)$$

$$T(n) = 5T(n/4) + n \quad (2)$$

$$T(n) = 7T(n/7) + n \quad (3)$$

$$T(n) = 9T(n/3) + n^2 \quad (4)$$

$$T(n) = 8T(n/2) + n^3 \quad (5)$$

$$T(n) = T(n-1) + 2 \quad (6)$$

$$T(n) = T(n-1) + n^c \quad (7)$$

$$T(n) = T(n-1) + c^n \quad (8)$$

$$T(n) = 2T(n-1) + 1 \quad (9)$$

Esercizi

Ricerca in array ordinato

Determinare un algoritmo del tipo *divide et impera* per il seguente problema:

PROBLEMA RICERCA IN ARRAY ORDINATO

INPUT: Un array $A[1 : n]$ di elementi confrontabili ordinato rispetto alla relazione $\leq$.
Un elemento x .

OUTPUT: Determinare la prima posizione di x in A se è presente, altrimenti ritornare “Non presente”.

Determinare la complessità.

Determinare un algoritmo del tipo *divide et impera* per il seguente problema:

PROBLEMA RICERCA IN MATRICE ORDINATA

INPUT: Una matrice di $n \times n$ di elementi confrontabili rispetto alla relazione $\leq$. Gli elementi in ogni riga sono ordinati in modo crescente come anche gli elementi in ogni colonna.

Un elemento x .

OUTPUT: Determinare una posizione (i, j) in A di x se è presente, altrimenti ritornare “Non presente”.[†]

Determinare la complessità.

[†]Non si richiede di trovare tutte le posizioni di x se x è presente più volte.

Esercizi

Minimo in una matrice

Determinare un algoritmo del tipo *divide et impera* per il seguente problema:

PROBLEMA MINIMO IN UNA MATRICE

INPUT: Una matrice di $n \times n$ di elementi confrontabili.

OUTPUT: Determinare in quale posizione è presente il minimo elemento nella matrice.

Determinare la complessità.